Лекция 10. Рекуррентные нейронные сети. Трансформеры

Денис Деркач, Дмитрий Тарасов

Использовались слайды Антона Кленицкого, Лены Войта

LAMBDA • HSE

20 апреля 2026 года

Напоминания

Представление текста

Первый шаг - токенизация (разбиение текста на отдельные слова - токены)

Дальше нужно перевести слова в числовой вид

Хотим представить слова в виде векторов небольшой размерности, которые отражают их смысл, чтобы

- Близкие по смыслу слова имели похожие вектора
- Разные по смыслу слова имели непохожие вектора

Основная идея:

Слова, которые часто встречаются в схожих контекстах, имеют похожее значение (distributional hypothesis)

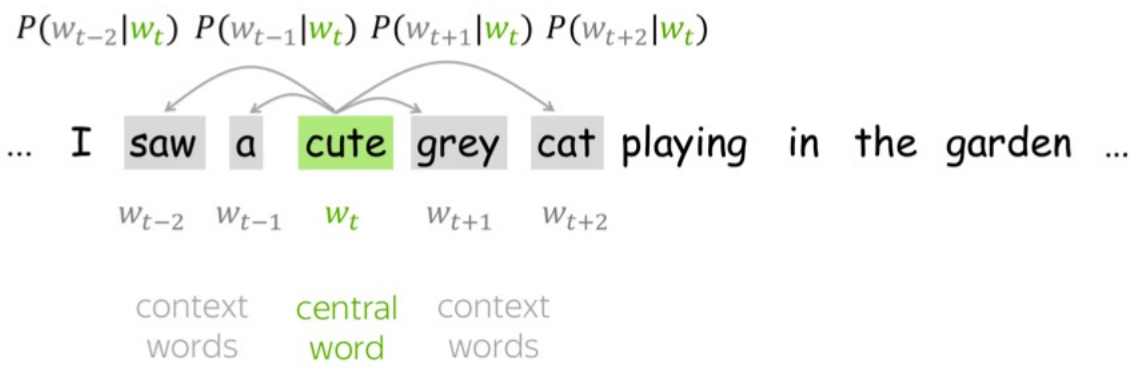
Word2Vec

Общая идея:

- Вектора слов, которые встречаются в одном контексте, должны быть близки

Реализация:

- Учимся предсказывать контекст (окружающие слова) по вектору данного слова

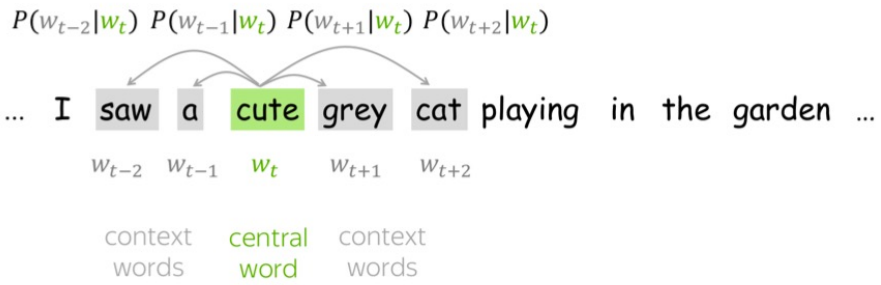


Word2Vec

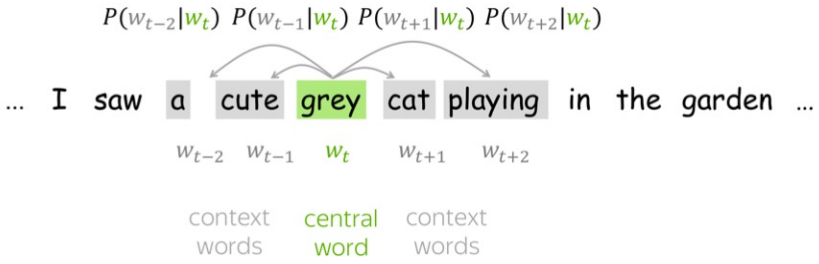
Алгоритм:

- Берем большой корпус текстов
- Проходим по текстам скользящим окном, смещаемся на одно слово на каждом шаге
- В каждом окне есть центральное слово и слова контекста (остальные слова в окне)
- Предсказываем вероятность окружающих слов на основе вектора центрального слова

1.



2.

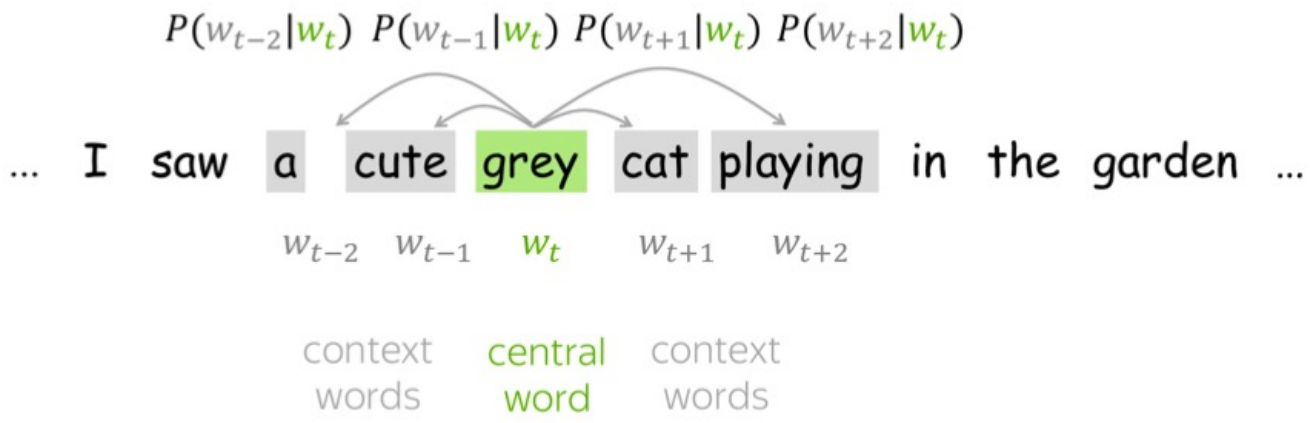


https://lena-voita.github.io/nlp_course/word_embeddings.html

Word2Vec – функция потерь

Функция потерь – отрицательное лог-правдоподобие:

$$L(\theta) = -\frac{1}{T}\sum_{t=1}^{T}\sum_{-m\le j\le m, j\ne 0}\log P(w_{t+j}|w_t,\theta)$$



Word2Vec – Обучение

Для каждого слова обучаются два вектора - v_w когда слово в центре и u_w когда слово в контексте.

$$P(o|c) = \frac{\exp(u_o^T v_c)}{\sum_{w \in V} \exp(u_w^T v_c)}$$

o - outside word, c - central word

Обучаем стохастическим градиентым спуском:

$$L_{t,j}(\theta) = -\log P(o|c) = -u_o^T v_c + \log \sum_{w \in V} \exp(u_w^T v_c)$$

$$v_c = v_c - \alpha \frac{\partial L_{t,j}(\theta)}{\partial v_c}$$

$$u_w = u_w - \alpha \frac{\partial L_{t,j}(\theta)}{\partial u_w}$$

Увеличиваем близость между v_c и u_o и уменьшаем близость между v_c и всеми остальными u_w.

Word2Vec – Negative Sampling

$$L_{t,j}(\theta) = -\log P(o|c) = -u_o^T v_c + \log \sum_{w \in V} \exp(u_w^T v_c)$$

Словарь очень большой, на каждом шаге SGD обновляем вектора всех слов u_w - долго.

Выход - сэмплирование отрицательных примеров

$$\sum_{w \in V} \exp(u_w^T v_c) \to \sum_{w \in \{w_{i_1}, .., w_{i_K}\}} \exp(u_w^T v_c)$$

Методы построения эмбеддингов

- Word2Vec: используйте, когда семантические связи имеют решающее значение, и у вас большой набор данных.
- GloVe: подходит для разнообразных наборов данных и когда важен охват глобального контекста.
- FastText: выбирайте морфологически богатые языки или когда обработка слов, не входящих в словарный запас, имеет решающее значение.

Word embeddings + нейросети

Subword Tokenization

Subword tokenization - баланс между представлением на уровне символов и представлением на уровне слов

На уровне символов

- Слишком длинные последовательности
- Теряется семантика, смысл

На уровне слов

- Проблема out-of-vocabulary words
- Слишком большой словарь

Subword Tokenization

- Часто используемые слова рассматриваются как отдельные токены
- Редкие слова раскладываются на несколько осмысленных подслов

В результате

- Ограниченный размер словаря
- Осмысленные представления
- Способность обработать любое новое слово
- Один токенизатор для всех языков сразу

Byte Pair Encoding

- Берем большой корпус текстов
- Заранее определяем размер словаря (например, 50к)
- Начинаем с отдельных символов как токенов
- На каждой итерации сливаем два токена, которые имеют максимальную частоту совместной встречаемости
- Заканчиваем, когда достигнут размер словаря или максимальная частота равна 1

AABABCABBAABAC

AA – 2
AB – 4 AB = D
BA – 3
BC – 1
CA – 1
BB – 1
AC – 1

ADDCDBADAC

AD – 2 AD = E
DD – 1
DC – 1
CD – 1
DB – 1
DA – 1
AC – 1

EDCDBEAC

Рекуррентные нейронные сети

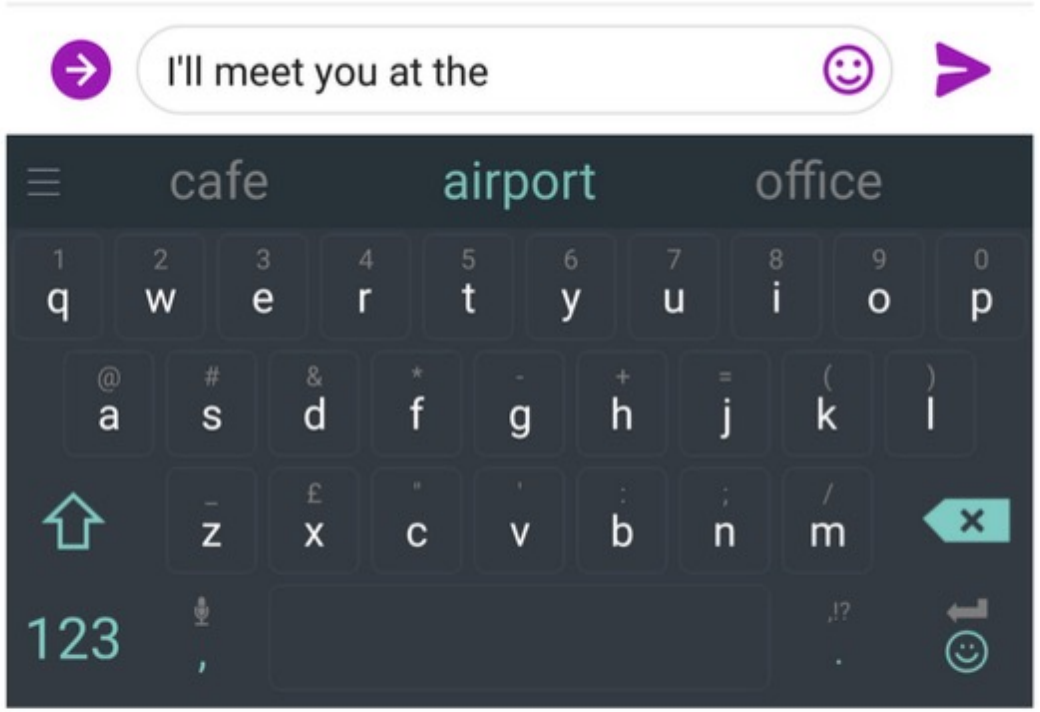
Типы данных

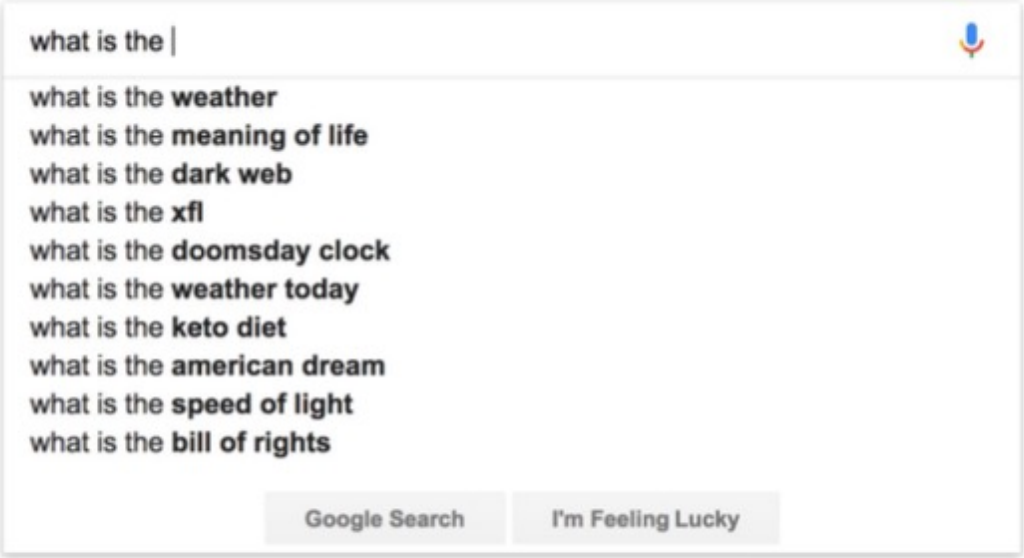
Часто нам встречаются последовательности данных

- Тексты: понимание речи, генерация текста
- Аудио
- Временные ряды
  - Анализ финансовых данных: фондовый рынок, сырьевые товары, Forex
  - Здравоохранение: частота пульса, уровень сахара (от медицинского оборудования и носимых устройств)
- Последовательности нуклеотидов в ДНК
- Пространственно-временные данные:
  - Самоуправление и отслеживание объектов
  - Тектоническая активность плит

Language model

Language model (языковая модель) – предсказание следующего слова по предыдущим





Language model

Формально - дана последовательность слов $x_1, x_2, ..., x_t$, нужно вычислить вероятность следующего слова x_{t+1}:

$$P(x_{t+1}|x_1, ..., x_t)$$

С помощью языковой модели можно назначить вероятность любому отрывку текста:

$$\begin{aligned} P(x_1, ..., x_t) &= P(x_1) \times P(x_2|x_1) \times ... \times P(x_t|x_1, ...x_{t-1}) \\ &= \prod_{t=1}^{T} P(x_t|x_1, ..., x_{t-1}) \end{aligned}$$

Language model

Классический вариант - подсчет n-грамм.

$$P(\text{books}|\text{the students opened their}) =$$
$$= \frac{P(\text{the students opened their books})}{P(\text{the students opened their})} \approx$$
$$\approx \frac{count(\text{the students opened their books})}{count(\text{the students opened their})}$$

Fixed-window neural net

output distribution

$$\hat{\boldsymbol{y}} = \text{softmax}(\boldsymbol{U}\boldsymbol{h} + \boldsymbol{b}_2) \in \mathbb{R}^{|V|}$$

hidden layer

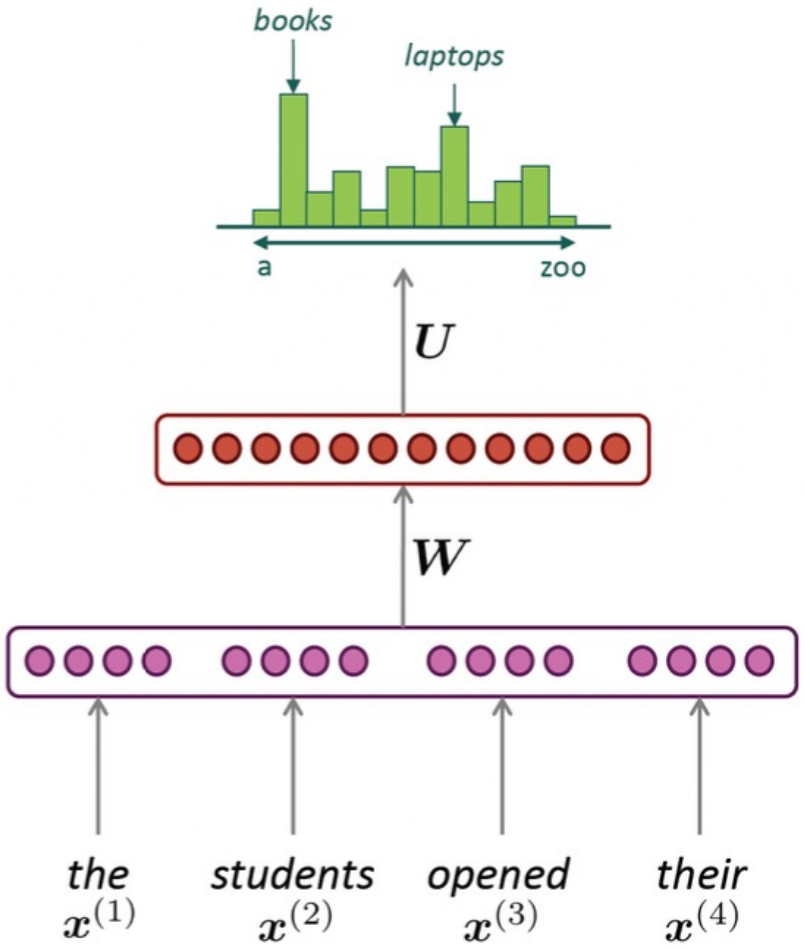
$$\boldsymbol{h} = f(\boldsymbol{W}\boldsymbol{e} + \boldsymbol{b}_1)$$

concatenated word embeddings

$$\boldsymbol{e} = [\boldsymbol{e}^{(1)}; \boldsymbol{e}^{(2)}; \boldsymbol{e}^{(3)}; \boldsymbol{e}^{(4)}]$$

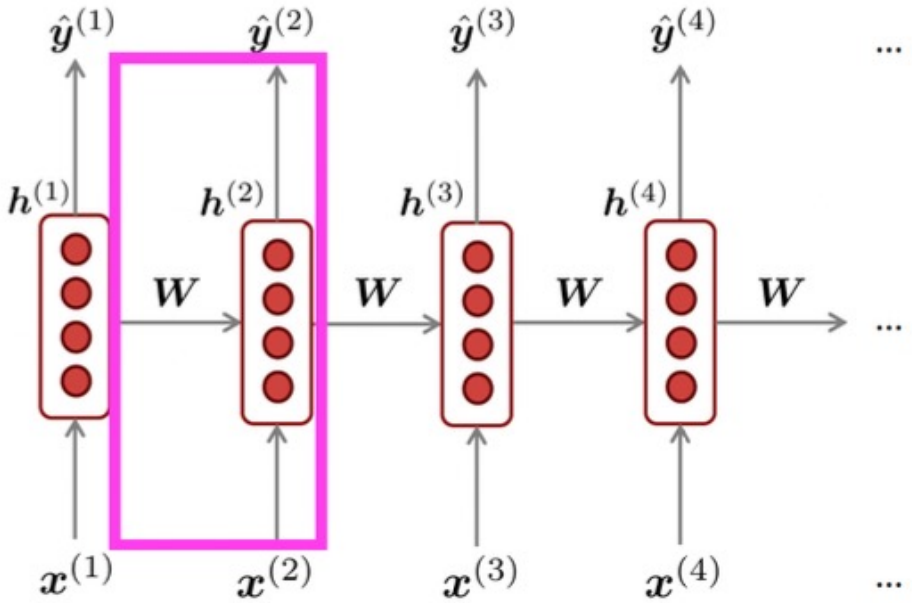
words / one-hot vectors

$$\boldsymbol{x}^{(1)}, \boldsymbol{x}^{(2)}, \boldsymbol{x}^{(3)}, \boldsymbol{x}^{(4)}$$



https://web.stanford.edu/class/cs224n/

Скрытые состояния

Хотим, чтобы модель обрабатывала последовательности любой длины

- Сохраняем вектор скрытого состояния (hidden state), передаем его на следующий шаг
- Weights sharing - применение одних и тех же весов на каждом шаге